

Métodos de Ordenamiento y Búsqueda

Documentación del código — Python

1. Descripción General

El programa implementa cuatro algoritmos de ordenamiento y dos de búsqueda. Está organizado en funciones independientes, una por algoritmo, y cuenta con un menú interactivo que permite al usuario generar datos aleatorios, comparar dos algoritmos de ordenamiento y ejecutar búsquedas cronometradas.

2. Estructura del Programa

Sección	Qué hace
<code>generar_datos(n)</code>	Genera N números enteros aleatorios
COMPLEJIDAD	Diccionario con Big O de cada algoritmo
<code>bubble_sort / insertion_sort</code> <code>selection_sort / merge_sort</code>	Algoritmos de ordenamiento
<code>busqueda_lineal / busqueda_binaria</code>	Algoritmos de búsqueda
<code>medir_tiempo(func, *args)</code>	Cronometra cualquier función
<code>menu_ordenamiento(datos)</code>	Menú para elegir y comparar dos algoritmos
<code>menu_busqueda(datos)</code>	Ejecuta ambas búsquedas automáticamente
<code>menu_principal()</code>	Menú principal que controla el flujo

3. Generación de Datos

La función **generar_datos(n)** recibe el tamaño N y devuelve una lista de N enteros aleatorios. El rango de valores es de 1 a N×10, de modo que escala con el tamaño del arreglo.

```
def generar_datos(n):  
    return [random.randint(1, n * 10) for _ in range(n)]
```

El usuario puede ingresar cualquier N (100, 1000, 10000). Los números se muestran en pantalla tal como se generaron, sin ningún orden.

4. Algoritmos de Ordenamiento

Todos los algoritmos reciben el arreglo original, hacen una copia interna (**a = arr[:]**) para no modificarlo, cuentan sus pasadas y devuelven el arreglo ordenado junto con el conteo de pasadas.

4.1 Bubble Sort

Compara pares de elementos vecinos e intercambia los que estén en orden incorrecto. Repite el proceso N veces. En cada pasada el elemento más grande 'burbujea' hacia el final.

```

for i in range(n): # pasada exterior
    for j in range(0, n-i-1):
        if a[j] > a[j+1]:
            a[j], a[j+1] = a[j+1], a[j]

```

4.2 Insertion Sort

Toma cada elemento desde la posición 1 y lo inserta en la posición correcta dentro de la parte izquierda ya ordenada, desplazando elementos hacia la derecha mientras sean mayores.

```

for i in range(1, len(a)):
    key = a[i]
    while j >= 0 and a[j] > key:
        a[j+1] = a[j] # desplaza hacia la derecha

```

4.3 Selection Sort

En cada pasada recorre todo el subarreglo no ordenado para encontrar el elemento mínimo y lo coloca en su posición definitiva al inicio. Siempre realiza N pasadas sin importar el estado del arreglo.

```

for i in range(n):
    min_idx = i
    for j in range(i+1, n):
        if a[j] < a[min_idx]: min_idx = j
    a[i], a[min_idx] = a[min_idx], a[i]

```

4.4 Merge Sort

Algoritmo recursivo. Divide el arreglo en dos mitades, ordena cada mitad recursivamente y luego las fusiona en orden. Es el más eficiente de los cuatro. Las pasadas se cuentan en cada llamada recursiva con un acumulador compartido (**_pasadas**).

```

mid = len(arr) // 2
left = merge_sort(arr[:mid], _pasadas)
right = merge_sort(arr[mid:], _pasadas)
# luego fusiona left y right en orden

```

5. Tabla de Complejidades (Big O)

Algoritmo	Mejor caso	Caso promedio	Peor caso
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$ — orden inverso
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$ — orden inverso
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$ — siempre igual
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$ — siempre igual

6. Algoritmos de Búsqueda

Ambas búsquedas se ejecutan automáticamente al elegir la opción 3 del menú. Se elige un valor aleatorio del arreglo como objetivo y se mide el tiempo de cada método.

6.1 Búsqueda Lineal

Recorre el arreglo elemento por elemento desde el inicio hasta encontrar el objetivo. Opera sobre el arreglo sin ordenar. En el peor caso recorre todos los elementos: **$O(n)$** .

```
for i, val in enumerate(arr):  
    if val == objetivo: return i
```

6.2 Búsqueda Binaria

Requiere que el arreglo esté ordenado. Divide el rango de búsqueda a la mitad en cada paso comparando el elemento central con el objetivo. Mucho más rápida: **$O(\log n)$** .

```
while izq <= der:  
    mid = (izq + der) // 2  
    if arr[mid] == objetivo: return mid  
    elif arr[mid] < objetivo: izq = mid + 1  
    else: der = mid - 1
```

7. Cronometraje

La función **medir_tiempo** es genérica: recibe cualquier función y sus argumentos, registra el tiempo antes y después de ejecutarla usando **time.perf_counter()** (el reloj de mayor precisión disponible en Python, con resolución de nanosegundos) y devuelve el resultado junto con el tiempo transcurrido en segundos.

```
def medir_tiempo(func, *args):  
    inicio = time.perf_counter()  
    resultado = func(*args)  
    fin = time.perf_counter()  
    return resultado, fin - inicio
```

8. Menú Principal

El menú principal tiene 4 opciones. El programa no permite ordenar ni buscar sin haber generado datos primero.

Opción	Acción
1	Genera N números aleatorios y los muestra en pantalla
2	Pide elegir dos algoritmos de ordenamiento y muestra resultados de cada uno
3	Ejecuta Búsqueda Lineal y Binaria automáticamente y muestra tiempos
4	Termina el programa